



Sommerprojekt Medientechnik

Strawberry Garden Café





Ideen & Settings

Elevator Pitch



Die Idee

Ein liebevoll gestaltetes Café-Webgame, in dem Spieler:innen Getränke und Snacks zubereiten, Bestellungen erfüllen und ihr Café Schritt für Schritt erweitern.



Das Ziel

Ein ästhetisches Spielerlebnis mit klarer Bedienung, motivierender Progression.





Spielmechanik & Features

Gameplay



Level 1



Tee

Level 2



Matcha

Level 3



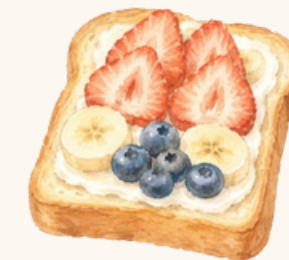
Kaffee

Level 4



Cake Cups

Level 5



Toast



Zufällige
Bestellungen



Coin-System



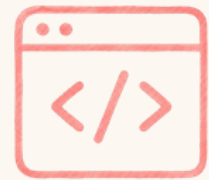
Freischaltbare
Inhalte



Visuelles
Feedback



Technische Umsetzung



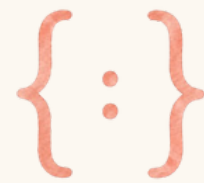
HTML

Struktur für Screens,
Karten und Interface



CSS

Layout, Farben,
Animationen



JavaScript

Spiellogik,
Bestellungen, Coins
und Interaktionen



Daten/JSON

Menüs, Bestellungen
und Levelinhalte



Bestellung



Auswählen



Prüfen



Belohnung



Freischaltung





Code



Level freischalten

```
const levelRequirements = {
  1: 0,
  2: 20,
  3: 50,
  4: 100,
  5: 150
};

let playerCoins = 0;
let unlockedLevels = [1];

function addCoins(amount) {
  const gained = Number(amount) || 0;
  playerCoins += gained;

  saveCoins();
  checkLevelUnlocks();
  updateCoinDisplay();

  if (gained > 0) {
    playSound("coins");
  }
}
```

→ Level-Anforderungen festlegen

→ Coins zählen

→ Belohnung hinzufügen

→ Unlock prüfen

→ Feedback auslösen

Level freischalten

```
function showLevelUnlockEffect(levelId) {  
  playSound("levelUnlock");  
  
  const notification = document.createElement("div");  
  notification.className = "level-up-popup";  
  
  notification.innerHTML = `  
    <div class="level-up-inner">  
      <div class="level-up-sparkles">💎 ✨ 💎</div>  
        
      <h2>LEVEL UP!</h2>  
      <p>Level ${levelId} unlocked</p>  
    </div>  
  `;  
  
  document.body.appendChild(notification);  
  
  setTimeout(() => notification.classList.add("show"), 20);  
  setTimeout(() => {  
    notification.classList.remove("show");  
    setTimeout(() => notification.remove(), 300);  
  }, 2800);  
}
```



Sound abspielen



Popup erstellen



Inhalt einfügen



Popup anzeigen



Automatisch entfernen

Level freischalten



```
function checkLevelUnlocks() {  
  for (let levelId = 1; levelId <= 5; levelId++) {  
    let requiredCoins = levelRequirements[levelId];  
  
    if (playerCoins >= requiredCoins && !isLevelUnlocked(levelId)) {  
      unlockedLevels.push(levelId);  
      showLevelUnlockEffect(levelId);  
    }  
  }  
  
  unlockedLevels.sort((a, b) => a - b);  
  saveUnlockedLevels();  
}  
  
function isLevelUnlocked(levelId) {  
  return unlockedLevels.includes(levelId);  
}
```



Alle Level durchgehen



Benötigte Coins lesen



Coins vergleichen



Fortschritt speichern

Interaktion



```
const cup = document.getElementById("coffee-cup-drag");
const machineScene = document.querySelector(".coffee-scene");

cup.addEventListener("dragstart", function (event) {
  playSound("dragPickup");
  event.dataTransfer.effectAllowed = "move";
  event.dataTransfer.setData("text/plain", "coffee-cup-drag");
  cup.classList.add("dragging");
});

machineScene.addEventListener("dragover", function (event) {
  event.preventDefault();
  machineScene.classList.add("picture-hover");
});

machineScene.addEventListener("drop", function (event) {
  event.preventDefault();
  playSound("dragDrop");

  coffeeState.cupPlaced = true;
  cup.style.display = "none";

  document.getElementById("coffee-machine-img").src =
    "images/coffeeMaschineXEmptyCup.png";

  show(document.getElementById("coffee-fill-btn"));
});
```

→ Elemente auswählen

→ Drag starten

→ Drop erlauben

→ Ablegen erkennen

→ Spielstatus ändern

→ Nächster Schritt erscheint

Interaktion

```
card.addEventListener("dragstart", e => {
  playSound("dragPickup");
  e.dataTransfer.setData("text/plain", card.id);
});

target.addEventListener("drop", e => {
  e.preventDefault();
  playSound("dragDrop");

  const id = e.dataTransfer.getData("text/plain");
  const card = document.getElementById(id);

  if (!card || card.dataset.used === "yes") {
    return;
  }

  cakeState.layers.push(card.dataset.layer);
  card.dataset.used = "yes";
  card.draggable = false;

  target.appendChild(card);

  if (cakeState.layers.length === 5) {
    setTimeout(() => showCakeStep(3), 500);
  }
});
```

→ Layer ziehen

→ Drop erkennen

→ Doppelte Nutzung verhindern

→ Reihenfolge speichern

→ Karte fixieren

→ Nächster Schritt

Interaktion

```
const guestImages = [
  "images/cat.png",
  "images/bunny.png",
  "images/bear.png",
  "images/dog.png"
];

function showNextOrder() {
  const catImg = document.querySelector(".game-cat");

  if (catImg) {
    let guest =
      guestImages[Math.floor(Math.random() * guestImages.length)];

    catImg.src = guest;
  }

  currentOrder = getRandomOrder();
  safeText("order-text", currentOrder.text);
}
```

→ Gastbilder speichern

→ Zufall auswählen

→ Gast anzeigen

→ Neue Bestellung laden



Demo





Vielen Dank für eure Aufmerksamkeit!

